

Basiswissen Softwaretest (Linz/Spillner)

Zusammenfassung (in Arbeit)

Patrick Bucher

04.09.2025

Inhaltsverzeichnis

1	Einleitung	2
1.1	Kapitelübersicht	2
2	Grundlagen des Softwaretestens	3
2.1	Begriffe und Motivation	3
2.1.1	Fehlerbegriff	4
2.1.2	Testbegriff	5
2.1.3	Grundsätze des Testens	6
2.2	Softwarequalität: Qualitätsmanagement und Qualitätssicherung	7
2.3	Der Testprozess	7
2.3.1	Testplanung	7
2.3.2	Testüberwachung und Teststeuerung	8
2.3.3	Testanalyse	8
2.3.4	Testentwurf	9
2.3.5	Testrealisierung	9
2.3.6	Testdurchführung	9
2.3.7	Testabschluss	10
3	Testen im Softwareentwicklungslebenszyklus	10
3.1	Sequenzielle Entwicklungsmodelle	11
3.1.1	Das Wasserfallmodell	11
3.1.2	Das V-Modell	11
3.2	Iterativ-inkrementelle Entwicklung	12
3.2.1	Klassische iterativ-inkrementelle Entwicklung	12
3.2.2	Agile Softwareentwicklung	12
3.2.3	Softwareentwicklung im Projekt- und Produktkontext	14
4	Statischer Test	15

5	Dynamischer Test	15
6	Testmanagement	15
6.1	Testorganisation	15
7	Testwerkzeuge	17

1 Einleitung

Software ist heutzutage allgegenwärtig und sie trägt nicht nur zum Funktionieren unserer Welt bei, von ihr hängt auch immer mehr unsere Sicherheit ab. Nicht nur die Abwicklung von Geschäftsprozessen hängt von Software ab, sondern ihre Erweiterbarkeit gibt auch vor, wie schnell eine Firma ihre Geschäftstätigkeit ausbauen kann. Die Qualität von Software ist ein entscheidender Faktor für den Erfolg von Produkten und Firmen.

Systematisches Testen von Software hilft Unternehmen dabei, die Qualität ihrer Softwaresysteme zu erhöhen. Das vorliegende Buch stellt das hierzu notwendige Grundlagenwissen bereit. Es richtet sich an Tester und Entwickler – sowie an alle, die im Rahmen der agilen Softwareentwicklung Testaufgaben übernehmen. Es richtet sich an Lehrende und Lernende gleichermaßen.

Das *International Software Testing Qualifications Board* (ISTQB) koordiniert die Zertifizierung im Bereich Software-Qualitätssicherung im Rahmen des ISTQB-Schemas und wird durch länderspezifische Gremien (wie z.B. durch das *Swiss Testing Board*) ergänzt. Die drei Ausbildungsstufen *Foundation*, *Advanced* und *Expert* werden dabei um zusätzliche Spezialistenmodule (u.a. fürs Testen im agilen Kontext) ergänzt.

1.1 Kapitelübersicht

Das vorliegende Buch deckt den Stoff bis zur ersten Stufe (*Foundation*) ab und behandelt in den folgenden Kapiteln diese Themen:

- **Kapitel 2** erörtert die Grundlagen des Softwaretests. Neben dem *Warum*, dem *Wann*, dem *Wozu* und dem *Wie* wird auf das Konzept des Testprozesses und auf die notwendigen Kompetenzen beim Testen eingegangen.
- **Kapitel 3** erläutert die Rolle des Testens in verschiedenen Entwicklungsmodellen (sequentiell, agil), die verschiedenen Teststufen und -arten, die Unterschiede zwischen funktionalen und nicht-funktionalen Tests, Regressionstests und Ansätze zur Verbesserung der Testautomatisierung.
- **Kapitel 4** behandelt statische Testverfahren, bei denen das Testobjekt nicht ausgeführt wird.
- **Kapitel 5** erörtert dynamische Tests und deren Einordnung in *Blackbox*- und *Whitebox*-Verfahren mit den dazugehörigen Testverfahren und -methoden.

- **Kapitel 6** behandelt die Organisation des Testprozesses und die dazu notwendigen Qualifikationen der involvierten Mitarbeiter. Nebst den Elementen einer Teststrategie werden auch Verfahren zur Aufwands- und Kostenschätzung des Softwaretests erläutert. Risikobasiertes Testen, Fehler- und Konfigurationsmanagement und Wirtschaftlichkeit sind ebenfalls Themen dieses Kapitels.
- **Kapitel 7** stellt verschiedene Arten von Testwerkzeugen vor und gibt Hinweise zu deren Auswahl und Einführung.

2 Grundlagen des Softwaretestens

In diesem Kapitel werden die Grundbegriffe des Softwaretestens eingeführt, etablierte Grundsätze des Testens vorgestellt und die Aktivitäten des Testprozesses erläutert.

2.1 Begriffe und Motivation

Industriell hergestellte Produkte werden zumeist durch Stichproben geprüft, was bei Softwareprodukten, die immateriell sind, nicht gleich funktioniert. Fehler in Software kosten nicht nur Zeit und Geld, sondern können auch den Ruf einer Organisation schädigen oder im Extremfall sogar zum Tod von Menschen führen.

Durch das Testen von Software kann deren Qualität eingeschätzt werden, und das Risiko unentdeckter Fehler, die sonst erst im Produktiveinsatz der Software zutage treten würden, kann minimiert werden. Beim Testen von Software sollen alle Beteiligten des Projekts involviert sein. Beim – statischen und dynamischen – Testen von Softwarekomponenten werden deren Fehler (genauer: Fehlerzustände bzw. Fehlerwirkungen) erkannt.

Beim *dynamischen* Testen kommt das *Testobjekt* (d.h. die Software) zur stichprobenartigen Ausführung, wozu das Testobjekt mit *Testdaten* versehen und einzelne Testfälle darauf ausgeführt werden, wonach geprüft wird, ob das beobachtete Ergebnis den Anforderungen entspricht.

Der gesamte Testprozess umfasst jedoch noch viele weitere Aktivitäten wie z.B. das Planen des Testvorgangs; das Abschätzen des Testaufwands; Analyse, Design und Umsetzung der Tests; das Erstellen von Berichten über Testfortschritt, Testergebnisse, Qualitätsbeurteilung und Risikobewertung.

Die Aktivitäten und Dokumentationen werden i.d.R. zwischen Auftraggeber und -nehmer ausgehandelt und unterliegen teilweise gesetzlichen Vorgaben oder Standards. Die Testaktivitäten unterscheiden sich im Lebenszyklus der Software; oftmals markieren Tests den Übergang von einer Phase in die nächste (z.B. die Freigabe einer neuen Version).

Obwohl Testaktivitäten von Werkzeugen abhängen, ist das Testen v.a. eine intellektuelle Tätigkeit, die Fachwissen und verschiedene Fähigkeiten erfordert. Neben der ausführbaren Software können im Rahmen von statischen Tests auch andere Artefakte wie z.B. Dokumentation, Anforderungen und Quellcode Testobjekte sein.

Je früher Fehler gefunden werden (z.B. bereits in den Anforderungen), desto besser ist das für den weiteren Entwicklungsprozess. Beim Testen wird auch geprüft, ob sich das System gemäss den Wünschen und Vorstellungen der Benutzer verhält. Es ist sinnvoll aber nicht immer machbar, die Benutzer im Rahmen einer Validierung möglichst im gesamten Entwicklungszyklus zu involvieren.

Ab einer gewissen Komplexität gibt es praktisch keine Softwaresysteme, die völlig fehlerfrei wären, da bei diesen oft Ausnahmen, Randbedingungen und Eingabekonstellationen nicht vollständig berücksichtigt werden können. Dennoch gibt es Software, die über eine lange Zeit zuverlässig funktioniert. Selbst wenn beim Testen keine Fehler mehr zu Tage treten, heisst das noch nicht, dass die Software tatsächlich fehlerfrei sei.

2.1.1 Fehlerbegriff

Anhand der Anforderungen und weiteren Informationen wird die *Testbasis* bestimmt, welche das erwartete Verhalten beschreiben und als Grundlage für die Entscheidung dient, ob korrektes oder fehlerhaftes Verhalten vorliegt.

Ein *Fehler* ist somit eine festgestellte Abweichung zwischen dem festgelegten Sollverhalten und dem beobachteten Istverhalten. Solche Fehler entstehen nicht durch Alterung oder Verschleiss, sondern sind vom Zeitpunkt der Entwicklung an Teil der Software, auch wenn sie erst später entdeckt werden.

Wird die Fehlfunktion für den Anwender oder Tester sichtbar, spricht man von einer *Fehlerwirkung* (engl. *failure*). Zwischen Ursache, die ihren Ursprung im *Fehlerzustand* (engl. *fault*) der Software hat, und dem Auftreten der Fehlerwirkung muss unterschieden werden.

Ein Fehlerzustand kann durch andere Fehlerzustände in der Software kompensiert werden, was als *Fehlermaskierung* bezeichnet wird. Durch die Korrektur eines maskierenden Fehlers können bisher verborgene Fehlerzustände an den Tag treten. Ein Fehlerzustand kann erst verspätet oder andernorts in der Software zu einer Fehlerwirkung führen, etwa bei der Verfälschung gespeicherter Daten.

Fehlerzustände entstehen durch die *Fehlhandlung* einer Person (engl. *error*) und können verschiedene Gründe haben:

- Unvollkommenheit des Menschen
- hohe Komplexität von Aufgaben, Architektur, Design und Code
- hoher Zeitdruck
- Missverständnisse und Fehlinterpretationen der Anforderungen
- mangelnde Erfahrung oder Ausbildung der Beteiligten
- Ablenkung, Unkonzentriertheit und Müdigkeit

Eine Fehlhandlung einer Person führt zu einem Fehlerzustand im Programmcode, der zu einer Fehlerwirkung in der Software führt, die durch das Testen aufgezeigt werden soll.

Wird beim Testen eine Fehlerwirkung festgestellt, ohne dass ein Fehlerzustand im Testobjekt vorliegt, spricht man von einem *falsch positiven Ergebnis* (engl. *false positive result*). Wird bei vorhandenem Fehlerzustand keine entsprechende Fehlerwirkung entdeckt, liegt ein *falsch negatives Ergebnis* (engl. *false negative result*) vor. Wird der Fehlerzustand durch eine Fehlerwirkung erkannt, ist das Ergebnis *richtig positiv*; liegt kein Fehlerzustand vor und wird auch keine Fehlerwirkung erkannt, ist das Ergebnis *richtig negativ*.

Durch die Analyse der zugrundeliegenden Fehlhandlungen zu aufgedeckten Fehlerzuständen können Erkenntnisse gewonnen werden, womit der Entwicklungsprozess verbessert und die Wiederholung der Fehlhandlung vermieden werden kann.

Da zunächst nur die Fehlerwirkung und nicht der ihr zugrundeliegende Fehlerzustand bekannt ist, muss die fehlerhafte Stelle in der Software zuerst lokalisiert werden. Diesen Vorgang bezeichnet man als *Debugging*. Beim Testen werden also Fehlerwirkungen aufgedeckt, beim Debugging werden die zugrundeliegenden Fehlerzustände lokalisiert.

Durch die Behebung des Fehlerzustands wird die Qualität der Software verbessert – sofern bei dieser Korrektur keine neuen Fehlerzustände eingebaut werden. Ein erneuter Test nach der Fehlerkorrektur wird als *Fehlernachtest* bezeichnet. Da bei der Fehlerkorrektur aber auch neue Fehler eingebaut werden können, die unter anderen Eingabekonstellationen eine Fehlerwirkung erzeugen, müssen noch weitere Tests durchgeführt werden, und nicht nur derjenige, der die Fehlerwirkung ursprünglich provozierte.

2.1.2 Testbegriff

Beim Testen werden verschiedene Ziele verfolgt:

- qualitative Bewertung von Arbeitsergebnissen
- Nachweis der Anforderungserfüllung
- Bereitstellen von Informationen zur Einschätzung der Qualität des Testobjekts
- Verringerung der Risiken durch Aufdeckung und Beschreibung von Fehlerwirkungen
- Analyse der Artefakte zur Vermeidung und Erkennung von Fehlerwirkungen
- Erhalten von Informationen zum Testabdeckungsgrad

Diese Ziele unterscheiden sich je nach Entwicklungsmodell (klassisch, agil) und Teststufe. Geht es etwa beim Komponententest um das Aufdecken von Fehlerwirkungen, steht beim Abnahmetest die Erfüllung der Nutzererwartungen im Vordergrund, und ob das Produkt zur Nutzung freigegeben werden kann.

Die *Testbasis* legt das Sollverhalten des Testobjekts fest und umfasst neben Anforderungsdokumenten, User-Stories oder Spezifikationen auch voraussetzbares Fachwissen und den gesunden Menschenverstand. Das Ausführen des Testobjekts mit bestimmten Testdaten bezeichnet man als *Testfall*. Nach einem solchen *Testlauf* wird geprüft, ob eine Fehlerwirkung – eine Abweichung des beobachteten vom erwarteten Ergebnis – vorliegt. Aus einer Testbasis werden *Testbedingungen* abgeleitet, welche je von einem oder mehreren Testfällen überprüft werden.

Ein Testobjekt kann nicht als Ganzes sondern nur unterteilt in verschiedene Testelemente geprüft werden. (Testfälle prüfen einzelne Testelemente.) Testfälle werden in *Testsuiten* zusammengefasst und so in einem *Testzyklus* ausgeführt. Der *Testausführungsplan* legt die zeitliche Ausführung der Testsuiten fest. Ein *Testskript* automatisiert die Ausführung einer Testsuite und kümmert sich dabei um die Einhaltung der Vor- und Nachbedingungen bzw. Vorbereitungs- und Aufräumarbeiten. Bei manuellen Tests wird ein entsprechender schriftlicher Testablauf zur Verfügung gestellt.

Die Ergebnisse der Testläufe werden im *Testprotokoll* gesammelt und im *Testbericht* zusammenfassend dokumentiert. Zu Beginn werden Auswahl von Testobjekten und -verfahren sowie Teilziele und Testberichtserstattung im *Testkonzept*; die Koordination der Testaktivitäten im *Testzeitplan* festgelegt.

Je nach Art des Projekts kann der Aufwand für das Testen einen mehr oder weniger grossen Anteil am Gesamtaufwand ausmachen. Konnte man bei klassischen Projekten das Verhältnis von Testern zu Entwicklern zur Einschätzung des Testaufwands beiziehen, fällt dies in agilen Projekten mit weniger starren Rollenverteilung schwer und kann höchstens anhand der Backlog-Aktivitäten grob abgeschätzt werden. Der Testaufwand muss ins Verhältnis zum Schadensausmass nicht gefundener Fehlerwirkungen und deren Eintretenswahrscheinlichkeit gestellt werden. ($\text{Fehlerkosten} = \text{Auftrittenswahrscheinlichkeit} \times \text{Schadensausmass}$)

Ans Testen sollte in einem Projekt so früh wie möglich gedacht werden (*Shift-Left-Ansatz*: die Testaktivitäten werden auf der Zeitachse nach links verlegt). Bereits beim Überprüfen von Anforderungen und beim Refinement von User Stories können beteiligte Personen mit Testwissen früh mögliche Fehler erkennen und so vermeiden.

Das Risiko grundsätzlicher Konstruktionsfehler kann durch Beteiligung von Testern in der Designphase reduziert werden. In der Umsetzungsphase können Tester beim Vermeiden fehlerhafter Testfälle behilflich sein. Durch die Verifikation vor der Freigabe kann die Wahrscheinlichkeit erhöht werden, dass der Kunde ein Produkt erhält, das seinen Anforderungen entspricht.

2.1.3 Grundsätze des Testens

Beim Testen haben sich in den letzten Jahrzehnten die folgenden Grundsätze etabliert:

1. Das Testen zeigt die Anwesenheit von Fehlerzuständen, kann aber nicht deren Abwesenheit beweisen, selbst wenn keine Fehlerwirkungen gefunden werden.
2. Ein vollständiges Testen ist nicht bzw. nur bei den trivialsten Testobjekten möglich; Tests sind immer nur Stichproben.
3. Frühes Testen spart Zeit und Geld, da früh erkannte Fehler oft einfacher zu beheben sind als solche, die sich erst etwa im Produktivbetrieb auswirken.
4. Fehler sind nicht gleichmässig über das ganze System verteilt sondern treten gehäuft in wenigen Komponenten auf.

5. Testfälle müssen laufend erweitert werden, um neue Fehlerwirkungen erkennen zu können. Wiederholtes Ausführen bestehender Testfälle kann nur Regressionsfehler aufdecken.
6. Testen ist kontextabhängig und muss dem zu prüfenden System angepasst werden. Keine zwei Systeme können genau gleich geprüft werden.
7. Ein System kann unbrauchbar sein, selbst wenn keine Fehler darin gefunden werden. Es müssen auch Benutzbarkeit und Akzeptanz des Benutzers gegeben sein, am besten indem man diese früh in die Entwicklung miteinbezieht.

2.2 Softwarequalität: Qualitätsmanagement und Qualitätssicherung

Das *Qualitätsmanagement* umfasst organisatorische Tätigkeiten und Massnahmen der Lenkung und Leitung einer Organisation in Qualitätsfragen. Das Qualitätsmanagement ist Sache des Managements. Dieses legt Arbeitsprozesse fest, deren Einhaltung im Rahmen der *Qualitätssicherung* durch die jeweiligen Projektbeteiligten geprüft wird.

Testen wird oft mit Qualitätssicherung gleichgesetzt, ist aber nur eine Massnahme davon. Die Qualitätssicherung umfasst alle Tätigkeiten, womit die Qualität einer Komponente oder eines Systems bewertet wird.

Zur *Qualitätssteuerung* gehört auch die Analyse der Ursachen von Fehlern. In der Qualitätssicherung dienen Testergebnisse dazu, Verbesserungspotenzial im Prozess zu ermitteln; in der Qualitätssteuerung dienen sie zur Behebung von Fehlerzuständen.

2.3 Der Testprozess

Unabhängig vom gewählten Entwicklungsmodell müssen Testarbeiten in kleinere Arbeitsschritte gegliedert werden. Ein Testprozess besteht aus verschiedenen *Testaktivitäten*, welche unabhängig vom konkreten Projekt in einer *Teststrategie* festgelegt werden.

Die im folgenden beschriebenen Testaktivitäten müssen nicht streng sequenziell aufeinander folgen, sondern können sich zeitlich überlappen. Die Testüberwachung und -steuerung findet parallel zu den anderen sechs Testaktivitäten statt. In der agilen Softwareentwicklung finden diese Testaktivitäten kontinuierlich und iterativ statt.

2.3.1 Testplanung

Die Testarbeiten werden von Beginn des Softwareentwicklungsprojekts an geplant und im weiteren Verlauf regelmässig überprüft und wenn nötig angepasst. Auf Basis einer Teststrategie wird ein *Testkonzept* erarbeitet, welches den *Testprozess* beschreibt.

Neben den Testobjekten und den nachzuweisenden Qualitätsmerkmalen wird festgehalten, welche Testaktivitäten welche Testziele nachweisen sollen. Auch benötigte Ressourcen und eine Zeitplanung sind Teil der Testplanung. Es werden Kriterien festgelegt, wann mit dem Testen

angefangen werden kann (*Definition of Ready*) und wann eine Testaktivitäten als abgeschlossen gilt (*Definition of Done*).

Wie in jedem Konzept müssen auch mögliche Risiken aufgeführt werden. Informationen über die Testbasis und wie Änderungen daran sich auf die Testaktivitäten auswirken, gehören auch ins Testkonzept. Für die verschiedenen Tests sind die jeweiligen Teststufen aufzuführen. Im *Testzeitplan* werden Aktivitäten mit Start- und Endtermin sowie mögliche Abhängigkeiten zwischen diesen Tätigkeiten festgehalten.

2.3.2 Testüberwachung und Teststeuerung

Die laufenden Testaktivitäten werden kontinuierlich im Vergleich zur Planung beobachtet. Abweichungen werden gemeldet, damit Gegenmassnahmen ergriffen werden können um das Erreichen der Testziele zu gewährleisten. Die Testplanung wird dabei aktualisiert.

Die Überwachung orientiert sich an den festgelegten Endkriterien zu den einzelnen Testaktivitäten. Können die durchgeführten Tests die Endkriterien nicht erfüllen, müssen zusätzliche Tests entworfen und durchgeführt werden.

Im *Testfortschrittsbericht* wird der Testfortschritt im Vergleich zur Planung an die Stakeholder gemeldet. Beim Erreichen von Meilensteinen können auch *Testabschlussberichte* angefertigt werden. Alle Testberichte sollen zielgruppengerecht über den Testfortschritt mit allen relevanten Details informieren, wobei auch geplante und tatsächliche Aufwände und Ressourcennutzung ausgewiesen werden. Als Grundlage hierzu dienen Berichte von Mitarbeitern aber auch erhobene Zahlen sowie durch Werkzeuge erstellte Auswertungen.

2.3.3 Testanalyse

Hier wird ermittelt, was zu testen ist. Dazu werden aus der Testbasis testbare Merkmale des Testobjekts ermittelt und daraus *Testbedingungen* abgeleitet. Dabei orientiert man sich am geforderten *Überdeckungsgrad* (*Wie viel muss abgedeckt werden?*) und an den Risiken (*Wie soll priorisiert werden?*).

Die Testbasis soll hierzu genau überprüft werden, denn aus einer fehlerhaften oder widersprüchlichen Testbasis können keine sinnvollen Testbedingungen abgeleitet werden. Die durchzuführenden Tests müssen dann nachweisen, ob das Testobjekt diese Bedingungen erfüllt. Das Testobjekt muss über eine diesen Tests zugängliche Schnittstelle verfügen.

Es muss zwecks Nachverfolgbarkeit *bidirektional* festgehalten werden, welche Testbedingungen welche Anforderungen prüft bzw. welche Anforderungen durch welche Testbedingungen geprüft werden. Gedanken über zu verwendende Testverfahren können in dieser Phase auch hilfreich sein.

2.3.4 Testentwurf

In dieser Phase wird ermittelt, *wie* zu testen ist. Aus den Testbedingungen werden *Testfälle* abgeleitet. Ein *abstrakter Testfall* verwendet dazu Bedingungen, denen die Eingabewerte genügen müssen (z.B. $1000 \leq x < 1500$), während ein *konkreter Testfall* exakte Eingabewerte festlegt (z.B. $x = 1250$). Abstrakte Testfälle müssen vor der Testdurchführung konkretisiert werden, sind dafür aber besser für spätere Testzyklen wiederverwendbar.

Pro Testfall sind Ausgangssituation (Vorbedingung), einzuhaltende Randbedingungen (Invariante) und erwartete Ergebnisse (Endbedingung) inkl. erwarteter Seiteneffekte, wie z.B. veränderte persistente Daten, zu definieren. Für Testdaten müssen Anforderungen festgelegt werden. Die Sollwerte können aus der Testbasis abgeleitet oder teilweise auch anhand einer Umkehrfunktion (z.B. Test der Verschlüsselung durch Entschlüsselung) festgelegt werden.

Die Priorisierung und Verfolgbarkeit aus der Testanalyse kann nun auf einzelne Testfälle heruntergebrochen werden. Die *Testinfrastruktur* bestehend aus Testumgebung, Testwerkzeugen und evtl. Testarbeitsplätzen muss ermittelt und bereitgestellt werden. Die *Testumgebung* umfasst neben dem Testobjekt auch die dazu notwendige Hardware und teilweise auch weitere Hilfsmittel.

Anhand von *Überdeckungselementen* – aus Testbedingungen abgeleitete Eigenschaften unter Verwendung eines Testverfahrens – werden Kriterien festgelegt, ab wann ausreichend getestet worden ist – z.B. mindestens 50% durch Unit Tests abgedeckte Codezeilen.

2.3.5 Testrealisierung

Diese Phase wird oft mit dem Testentwurf kombiniert. Hier sind alle Aktivitäten soweit vorzubereiten, dass die Testfälle in der nächsten Phase ausgeführt werden können.

Testmittel und Testinfrastruktur müssen bereitgestellt und geprüft werden. Testdaten müssen in die Testumgebung übernommen und ebenfalls überprüft werden. Testfälle müssen konkretisiert, zu Testsuiten gruppiert und in eine sinnvolle Reihenfolge gebracht werden, sodass die Nachbedingung eines Testfalls möglichst als Vorbedingung seines Nachfolgers genutzt werden kann. Auch die Priorisierung der Testfälle ist dabei zu beachten.

Eine Testsuite soll auch die Aufräumarbeiten nach den durchgeführten Testfällen berücksichtigen. Testskripte können diese Schritte automatisieren. Im Testausführungsplan werden die definierten Abläufe festgehalten.

2.3.6 Testdurchführung

Nun gelangen die Testfälle gemäss *Testausführungsplan* zur Ausführung. Zunächst empfiehlt es sich, die Hauptfunktionen des Testobjekts im Rahmen eines *Smoke-Tests* zu überprüfen; sind diese bereits fehlerhaft, lohnt sich ein Weitertesten i.d.R. nicht.

Die ausgeführten Testfälle sind zu protokollieren, wozu folgende Angaben festgehalten werden: Testergebnis (bestanden, fehlgeschlagen oder blockiert, d.h. kann nicht ausgeführt werden), Tester, Testzeitpunkt und allenfalls Gründe für das Auslassen eines Testfalls.

Durch dieses Protokoll wird der Testvorgang für andere Parteien nachvollziehbar und die Umsetzung der gewählten Teststrategie kann damit nachgewiesen werden. Abweichungen zwischen erwarteten und tatsächlichen Testergebnissen werden ebenfalls protokolliert. Bei der Auswertung des Protokolls kann dann entschieden werden, ob eine Fehlerwirkung vorliegt. Beim Testen sollen auch die *Überdeckungsgrade* gemessen und protokolliert werden; bei Bedarf auch der Zeitverbrauch.

Mithilfe der Verfolgbarkeit – Testbasis, Testbedingungen, Testfälle, Testergebnisse – kann nun nachvollzogen werden, welche Anforderungen erfüllt, teilweise erfüllt oder nicht erfüllt sind.

2.3.7 Testabschluss

Der Zeitpunkt des Testabschlusses hängt vom verwendeten Entwicklungsmodell ab und kann auf die Freigabe einer Software oder eines Wartungsreleases, auf das Ende einer Iteration, auf den Abschluss eines Testprojekts oder auf einen sonstigen Meilenstein fallen.

Daten werden zusammengetragen, Erfahrungen ausgewertet und Testmittel zur Ablage gesichert (z.B. als Container-Images). Offene Fehler sind vollständig als Fehlerberichte gemeldet und werden in die nächste Iteration übernommen.

Die Testaktivitäten und deren Ergebnisse werden im *Testabschlussbericht* zusammengefasst und so den Stakeholdern zur Verfügung gestellt. Je nach Branche muss von Gesetzes wegen ein Nachweis über die durchgeführten Tests erbracht werden.

In den Testaktivitäten gemachte Erfahrungen (z.B. Abweichungen zwischen Plan und Umsetzung) werden zwecks Erkenntnisgewinn für spätere Iterationen oder andere Projekte analysiert, wodurch der Testprozess an Reife gewinnt.

3 Testen im Softwareentwicklungslebenszyklus

Ein Softwareentwicklungsprojekt orientiert sich an einem im Voraus festgelegten Vorgehensmodell. Damit werden die Aufgaben in eine logische Reihenfolge gebracht und auf Phasen oder Iterationen verteilt. Auch werden die Arbeiten auf verschiedene Rollen verteilt.

Verschiedene Vorgehensmodelle machen unterschiedliche Vorgaben zum Testen. Grundsätzlich unterscheidet man zwischen sequenziellen, iterativ-inkrementellen und agilen Entwicklungsmodellen.

3.1 Sequenzielle Entwicklungsmodelle

Der Prozess wird als ein sequenzieller Ablauf von Aktivitäten verstanden, nach deren Durchlauf am Ende das gewünschte Produkt in der geforderten Qualität fertig bereitsteht. Ein zeitliches Überschneiden der einzelnen Aktivitäten ist nicht vorgesehen. Zwischen Projektstart und Auslieferung können Monate oder Jahre vergehen.

3.1.1 Das Wasserfallmodell

Nach diesem Modell sind die einzelnen Phasen – *System Requirements, Software Requirements, Analysis, Program Design, Testing* und *Operation* – zeitlich streng voneinander getrennt. Das Testen wird als einmalige und den Entwicklungsarbeiten nachgelagerte Aktivität verstanden, nicht als projektbegleitende Tätigkeit.

3.1.2 Das V-Modell

Hier wird das Wasserfallmodell um ein erweitertes Verständnis der Testaktivitäten ergänzt. Zu jeder Entwicklungsarbeit – *Anforderungsdefinition, funktionaler Systementwurf, technischer Systementwurf, Komponentenspezifikation* – gibt es eine korrespondierende Testaktivität – *Abnahmetest, Systemtest, Integrationstest, Komponententest*.

Die Entwicklungsarbeiten bilden die absteigende Flanke, die Testarbeiten die aufsteigende – und unten in der Mitte steht das *Programmieren*. Geht man bei den Entwicklungsarbeiten vom Groben ins Feine («top-down»), setzt man bei den Testaktivitäten die einzelnen Teile wieder zu einem Ganzen zusammen («bottom-up»). Die Ergebnisse aus den Entwicklungsarbeiten werden dabei sukzessive integriert und folgendermassen getestet:

- *Komponententest*: Erfüllt der Baustein seine Spezifikation?
- *Integrationstest*: Spielen die Komponenten wie gewünscht zusammen?
- *Systemtest*: Erfüllt das System als Ganzes die spezifizierten Anforderungen?
- *Abnahmetest*: Erfüllt das System aus Kundensicht die vereinbarten Leistungsmerkmale?

Diese Testaktivitäten sind nicht als eine blosse zeitliche Unterteilung zu verstehen, sondern verfolgen unterschiedliche Testziele auf verschiedenen Abstraktionsebenen. Dabei werden unterschiedliche Testmethoden und Testwerkzeuge von für die jeweilige Testaktivität spezialisiertem Personal angewendet.

Die Testaktivitäten sind von unterschiedlichem Charakter:

- *Verifizierende* Tests prüfen, ob ein Testobjekt seine Aufgabe gemäss Spezifikation erfüllt.
- *Validierende* Tests prüfen, ob ein Testobjekt für seinen Einsatzzweck geeignet ist.

Mit steigender Teststufe nimmt der verifizierende Charakter der Testaktivitäten ab und der validierende zu.

Die vorbereitenden Testaktivitäten (Testplanung, Testanalyse, Testentwurf) können im V-Modell parallel zu den Entwicklungsarbeiten in der absteigenden Flanke erfolgen. Entwicklungs- und Testarbeiten werden einander in diesem Modell als gleichwertig gegenübergestellt. Auf jeder Teststufe wird gegen die korrespondierende Entwicklungsstufe getestet.

3.2 Iterativ-inkrementelle Entwicklung

In der Praxis trifft man kaum rein sequenzielle Entwicklungsprojekte an, die etwa nach einem Durchlauf des V-Modells abgeschlossen wären. Stattdessen werden solche Vorgehensmodelle oder Teile davon iterativ durchlaufen, woraus ein Produkt inkrementell entsteht.

3.2.1 Klassische iterativ-inkrementelle Entwicklung

Das Produkt wird schrittweise verbessert, wobei man sich auf Rückmeldungen des Kunden abstützt, dessen Bedürfnisse nach jeder Iteration besser erfüllt werden sollen. Erfahrungen aus vorhergehenden Iterationen dienen ebenfalls zur Verbesserung des Produkts.

Durch das schrittweise Vorgehen mit häufigen Releases wird die *Time to Market* verkürzt und vermieden, an den Bedürfnissen des Kunden vorbei zu entwickeln. Klassisch iterativ-inkrementelle Modelle wie z.B. der *Rational Unified Process* (RUP) sind heute nur noch sehr wenig verbreitet.

3.2.2 Agile Softwareentwicklung

In der agilen Softwareentwicklung wird das vorausplanende Projektmanagement durch eine adaptive Projektsteuerung ersetzt, womit schnell auf angepasste oder neue Kundenwünsche reagiert werden kann, ohne dabei Zeit zum Nachtragen der Projektdokumentation zu verlieren.

In Abgrenzung zu den klassischen schwergewichtigen und dokumentlastigen Modellen gelten agile Methoden wie *Extreme Programming*, *Kanban* und das am meisten verbreitete *Scrum* als leichtgewichtig, wobei Projekt- und Prozessdokumentation minimiert werden. Scrum zeichnet sich aus durch:

- *Sprints*: kurze Iterationen fester Länge
- *Product- & Sprint Backlog*: Priorisierungen der Kundenanforderungen
- *Timeboxing*: begrenzte Zeitfenster für Aufgaben und Besprechungen
- *Transparenz*: Offensichtlichmachung des Sprint-Fortschritts durch regelmässige Besprechungen (*Daily Scrum*) und jederzeit einsehbare Taskboards

Auf Basis des «Whole Team»-Ansatzes werden Aufgaben und Probleme bevorzugt gemeinsam von mehreren Teammitgliedern angegangen, wobei jedes Mitglied seine Stärken und sein Fachwissen einbringen kann. Anstelle einer starren Rolleinteilung – Programmierer, Tester – unterstützen sich die Teammitglieder gegenseitig, auch bei Testaufgaben.

Für die Qualität des resultierenden Produkts sind alle im Team gleichermaßen verantwortlich. Es gibt jedoch Projekte, in denen dieser Ansatz beispielsweise aus regulatorischen Gründen nicht gangbar ist, etwa wenn Entwicklungs- und Testteam aus sicherheitstechnischen Überlegungen voneinander getrennt agieren müssen.

Freigaben können mehrmals pro Jahr bis im Extremfall mehrmals täglich erfolgen. Damit eine so hohe Taktung der Releases mit der notwendigen Qualität überhaupt möglich ist, müssen die Testfälle grösstenteils automatisch bereitgestellt werden, sodass die Inkremente früherer Releases automatisch durch Regressionstests mitgeprüft werden können.

Dadurch nimmt die Anzahl der Testfälle von Iteration zu Iteration laufend zu. Nur dank hoher Testautomatisierung kann die wachsende Testmenge bei gleichbleibender Sprintdauer konsequent durchgeführt werden. Zur Beschleunigung der Testdurchläufe können Tests geringerer Priorität auch von der Ausführung ausgenommen werden.

In einem agilen Projekt werden die Anforderungen in einem iterativ-inkrementellen Prozess aufgenommen. Dabei steigt nicht nur die Menge der Anforderungen mit jeder Iteration an, sondern auch deren Detailgrad. Durch eine enge Zusammenarbeit der Stakeholder untereinander wird sichergestellt, dass beim Verständnis der Anforderungen keine Missverständnisse auftreten.

Dieses gemeinsame Verständnis soll über die Form der *User Story* erreicht werden, mit welcher Anforderungen folgendermassen beschrieben werden. (In der Praxis sind leicht unterschiedliche Satzschablonen in Gebrauch):

Als *ROLLE* möchte ich, dass *ZU ERREICHENDES ZIEL*, damit *RESULTIERENDER NUTZEN*.

Die Qualität der User Stories wird mit den sogenannten *INVEST*-Kriterien sichergestellt. Demnach soll eine User Story folgendes sein:

- Independent: *unabhängig* von anderen User Stories
- Negotiable: *verhandelbar* durch Spielraum bei der Umsetzung
- Valuable: *wertvoll/nützlich* für den Anwender bzw. Kunden
- Estimable: *abschätzbar* im Aufwand durch ausreichende Beschreibung
- Small: *klein* genug für eine Umsetzung ohne weitere Aufteilung
- Testable: *testbar* durch hinreichende Akzeptanzkriterien

Teammitglieder mit Testwissen können durch die Erfüllung des letztgenannten Kriteriums viel dazu beitragen, die Testkosten für eine Story tief zu halten.

Beim Erstellen einer User Story empfiehlt sich das *3C-Schema*:

1. Card: Die User Story wird auf einer physischen oder virtuellen Story-Karte festgehalten.
2. Conversation: In einem Dialog, der von Releaseplanung bis zur Umsetzung andauern kann, klären die Stakeholder die User Story inhaltlich.
3. Confirmation: Die korrekte Umsetzung der User Story wird explizit aufgrund der festgelegten Abnahmekriterien bestätigt.

Abnahmekriterien sind nichts weiteres als Testbedingungen, die beim Abnahmetest einer User Story zur Anwendung kommen. Diese tragen dazu bei, dass

- bei den Stakeholdern Konsens über die Interpretation der User Story herrscht,
- der Umfang der User Story klar eingegrenzt ist, und
- der Arbeitsaufwand einschätzbar und planbar ist.

Bei der *abnahmetestgetriebenen Entwicklung* (ATDD: engl. *acceptance test-driven development*) werden die Abnahmetestfälle bereits vor der Implementierung umgesetzt. Zuerst werden die Abnahmekriterien an einem Spezifikations-Workshop gemeinsam unter den Stakeholdern geklärt. Anschliessend setzt das Team daraus die Abnahmetestfälle um.

Die Testfälle sollten dabei nicht über die jeweilige User Story hinaus gehen und auch keine Überschneidungen untereinander haben, aber nicht nur den Normalfall, sondern alle möglichen Sonderfälle behandeln. Schwierigkeiten bei der Erarbeitung der Abnahmekriterien deuten auf eine unklare User Story hin. Das Team ist selber dafür verantwortlich, zusätzliche nicht-funktionale Test umzusetzen, wenn es diese als angebracht sieht.

3.2.3 Softwareentwicklung im Projekt- und Produktkontext

Je nach umzusetzendem Produkt oder Projekt unterscheiden sich die Anforderungen an Nachvollziehbarkeit und Planung, was einen Einfluss auf das auszuwählende Entwicklungsmodell hat. Hierbei können verschiedene Kriterien eine Rolle spielen:

- die angestrebte *Time to Market*
- das Einsatzgebiet (intern, bei Kunden) und die geplante Lebensdauer (vorübergehend, langfristiges Kundenangebot)
- das technische Umfeld (zentrale Web-Anwendungen, auf Offline-Geräten vorinstallierte Software)
- identifizierte Produktrisiken (geringe bei Unterhaltungsanwendungen, sehr hohe bei Medizinalsoftware)
- organisatorische und kulturelle Aspekte (eingespieltes lokales Team, geografisch verteilte Einzelkämpfer)

Vorgehensmodelle können miteinander kombiniert und auf die jeweiligen Bedürfnisse zugeschnitten werden (engl. «Tailoring»). Dies kann auch die Testaktivitäten betreffen, welche aber in jedem Fall folgenden Anforderungen genügen sollten:

- Testaktivitäten sollen schon früh im Projekt angegangen werden (Spezifikation von Testfällen, Aufbau der Testumgebung).
- Für jede Entwicklungsaktivität muss eine passende Testaktivität vorgesehen sein.
- Die Testaktivitäten müssen auf jeder Teststufe den Testzielen entsprechend ausgerichtet sein (z.B. mehr oder weniger Fokus auf die Validierung oder Verifizierung).
- Testanalyse und Testentwurf erfolgen bereits während der Entwicklung und nicht nachgelagert.
- Tester sind bereits beim Aufnehmen der Anforderungen und bei deren Prüfung (Review) involviert.

4 Statischer Test

TODO

5 Dynamischer Test

TODO

6 Testmanagement

In diesem Kapitel geht es um die organisatorischen Voraussetzungen für effizientes Testen.

6.1 Testorganisation

Es ist zwar verlockend einfach, die Entwickler ihre Software gleich selber testen zu lassen. Durch eine personelle Trennung von Umsetzung und Testen kann jedoch die «Blindheit gegenüber eigenen Fehlhandlungen» vermieden werden.

Ein unabhängiger Tester ist dem Testobjekt gegenüber weniger voreingenommen und findet dadurch mehr oder andersartige Fehlerwirkungen. Auch ist ein unabhängiger Testen weniger von impliziten Annahmen befangen, die ein Programmierer womöglich bei der Umsetzung trifft.

Andererseits kann eine solche Trennung die Zusammenarbeit und Kommunikation zwischen Testern und Entwicklern erschweren. Tester sind teilweise mit dem Testobjekt wenig vertraut und werden aufgrund knapper Ressourcen oft als Flaschenhals wahrgenommen. Ausserdem kann das Qualitätsbewusstsein der Entwickler leiden, wenn das Auffinden von Fehlerwirkungen an Tester abdelegiert wird.

Die folgenden fünf Modelle, angeordnet nach ansteigender Unabhängigkeit der Tester, soll obengenannte Vorteile mitbringen und dabei die genannten Nachteile reduzieren:

1. *Entwicklertest*: Ähnlich wie beim *Pair Programming* arbeiten zwei Entwickler zusammen, indem sie ihre Umsetzungen wechselseitig testen, wodurch die Blindheit gegenüber eigener Fehlhandlungen entfällt.
2. *Unabhängige Tester*: Es gibt separate, spezialisierte Tester im Team, welche die Testarbeiten ausführen.
3. *Unabhängige Testteams*: Die Organisation verfügt über dedizierte Testteams, die von Mitarbeitern anderer Teams zeitweise verstärkt werden können.
4. *Testspezialisten*: Aus einem Pool von Mitarbeitern mit spezialisiertem Testwissen kann von Projekten zeitweise Unterstützung angefordert werden.
5. *Testdienstleister*: Die Testarbeiten werden von einem externen Dienstleister ausserhalb (Outsourcing) oder innerhalb (Insourcing) des Unternehmens ausgeführt.

Die Wahl des Modells ist dabei je nach Teststufe zu beurteilen: Bei Komponententests sollte entwicklungsnahe nach den Modellen 1 oder 2 gearbeitet werden. Bei Integrations- und Systemtests ist eine höhere Perspektive sinnvoll, die man nach den Modellen 3, 4 und 5 erhält. Diese Modelle eignen sich auch beim V-Modell, nach welchem Entwicklungs- und Testarbeiten voneinander getrennt sind.

Bei agilen Vorgehensweisen wird oft Modell 1 bevorzugt. Andere Modelle gelten dort gar als der Agilität abträglich, was jedoch ein Trugschluss ist: Auf Testautomatisierung spezialisierte Teammitglieder oder Dienstleister können dabei helfen, die ständig wachsenden Anforderungen an die Testautomatisierung (etwa durch verbesserte *Continuous Integration*-Pipelines) besser zu bewältigen. Externe Spezialisten können als Coaches für Testaufgaben fungieren. Der Product Owner, unterstützt durch Mitarbeiter aus Fachabteilungen, kann als Spezialist für Abnahmetests angesehen werden.

Die Ausführung der Testarbeiten erfordert verschiedene Rollen mit Spezialwissen:

- Der *Testmanager* ist für die Testaktivitäten von Entwicklungsprojekten verantwortlich. In kleinen agilen Projekten kann das ein Teammitglied mit dem entsprechenden Testwissen sein. In grösseren Projekten und Organisationen ist dies ein Testexperte, teilweise mit Personal- und Führungsverantwortung. Ein Testmanager erarbeitet eine Teststrategie, fördert und entwickelt Tester, unterstützt Entwicklungsteams bei Testaufgaben, erstellt Testkonzepte in Zusammenarbeit mit Projekt-Stakeholdern, bereitet Tests vor und stellt die dazu nötigen Ressourcen zur Verfügung; realisiert, überwacht und steuert Testaktivitäten; unterstützt bei der Beschaffung und Einführung von Testwerkzeugen und führt unterstützende Prozesse zu den Testaktivitäten ein und begleitet diese.
- Der *Tester* ist für Testtätigkeiten zuständig und führt diese aus. Je nach Teststufe ist diese Rolle anders ausgeprägt: Als Entwickler für Komponenten- und Integrationstests oder als Business Analyst für Abnahmetests. Neben IT-Fachwissen ist auch Vertrautheit mit dem Testobjekt und Wissen über Testwerkzeuge erforderlich. Tester führen Reviews von Testdokumenten und -artefakten durch, stellen Testdaten bereit, wenden Testwerkzeuge an, führen manuelle Tests durch, protokollieren Testergebnisse und verfassen Fehlerberichte. Dazu benötigen Tester auch soziale Kompetenz und zahlreiche weitere «Soft Skills» wie etwa eine gute Kommunikationsfähigkeit.

Testteams müssen oft (zeitweise) durch weitere Spezialisten für Datenbanken, Netzwerke oder durch bestimmte Fachexperten ergänzt werden. Fehlen solche Ressourcen intern, können auch externe Dienstleister beigezogen werden.

7 Testwerkzeuge

TODO